Ian Foss
CS-499
Enhancement Two Narrative

The artifact I chose for this enhancement was again the Android application I created in my previous SNHU class, CS-360 Mobile Architecture and Programming. It is an inventory management app that supports persistent data for users and inventory items using the Room database. I chose this artifact for the enhancement because it already utilizes data structures and algorithms, but on a very basic level that can be improved well. The list of items is the perfect place to implement data structures and algorithms, as those aspects will directly affect how the list of items can be manipulated in the application. This is a common use case in the industry that can provide real value to users.

The artifact showcases my ability to work with list-based data. It demonstrates principles of algorithmic thinking, specifically with the searching and sorting algorithms that I implemented in the enhancement. Search logic was added that improved usability by allowing users to find an item in the inventory without scrolling through the list manually. Filtering logic for low stock or out of stock items in the inventory was also added. Finally, sorting logic was added to allow users to view items alphabetically or by quantity. Since the application uses the Room database, the SQL queries dictated the algorithmic complexity involved in fulfilling the functionality. The search and filter functionality, along with updating the RecyclerView with the altered list of items, needs to iterate through every item in the list at most once. This is because the search logic needs to find any item with a matching string, the filter logic needs to find items that meet the filter criteria, and the RecyclerView adapter needs to update the RecyclerView with the result set after the modifications have been made. From an algorithmic complexity standpoint, the worst-case scenario time complexity for these operations will be O(n), where n is the number of items in the inventory. For the sorting functionality, there is an increased operational complexity. The Room database sorting operations are abstracted away by SQLite after the ORDER BY query is passed to it. Generally speaking, the common complexity estimate for sorting can be described as O(n log n), which could be accomplished by the quicksort or merge sort algorithms that are commonly used. While these enhancements do perform more operations and utilize more memory than the original app, the efficiency for the user is greatly improved as they no longer have to manually scroll through the list to find an item.

I believe I did meet the course outcomes for this enhancement that were planned in module one. This enhancement strongly supports the course outcomes of designing and evaluating computing systems using algorithmic principles. The application uses a collection of inventory objects and applies searching, sorting, and filtering logic. This demonstrates how data structures can be used to store records, and how algorithms can be implemented to manipulate those records in useful ways. This enhancement also supports the course outcome of using well-founded computing techniques. The logic for this additional functionality was implemented through Room DAO queries and repository methods. This contributed to a separation of concerns between the activity/UI logic and the database operations. I do not have any updates to my coverage plans at this time.

This enhancement provided me with good learning experiences in a couple of ways. I learned how to use the RecyclerView in more depth. The inventory list needed to initially load every item but then display only the items that match the search, sort, or filter criteria. This required ensuring I was observing the most recent LiveData source and updating the adapter with the new list of items. One challenge was ensuring I was updated all the required classes to maintain the separation of concerns. It would have been a lot easier to just add all the logic in the inventory activity. Instead, to enhance the artifact using Android best practices, I needed to create new DAO queries to support the new functionality, repository methods to utilize the DAO queries, and then add logic for the UI components and repository method calls. Overall, this enhancement was a good learning experience in applying searching, sorting, and filtering logic to a real data set.